REPORT · 2026年10月18日

全链路缺陷排查

H3D v1.1.0 · r6.4 端到端复盘

文保数字化研发组
2026年10月18日

目 录

一、结论先行 3

1.1 九项缺陷一览 3

二、四项致命缺陷的证据链 4

2.1 B3：投影坐标错位 1.874 倍（首因） 4

2.2 B1：交付包只有 .obj，没有材质和贴图 4

2.3 B2：GLB 静默无纹理 4

2.4 B4：照片按 float64 全量常驻内存 5

三、次级缺陷与修复 6

3.1 B5：平铺照片目录 6

3.2 B6：场景② 唯一几何源被无端粗化 6

3.3 B7：对同坐标系的网格再做 ICP 6

3.4 B8：无色彩数据导出黑色模型 6

3.5 B9：贴图命名双份 6

四、修改清单 7

五、验证 8

5.1 单元测试 8

5.2 真实数据核对 8

5.3 端到端（内置示例，16 张 640 px） 8

六、仍未解决 / 需注意 9

七、下一步 10

一、结论先行

前几轮修的都是**「跑不完」**：超时、并发互删、界面假死、整尊失败。但本轮按链路逐段核对后发现，即使管线**全部阶段显示完成**，交付出来的东西仍然是不可用的——问题出在链路**末端**：颜色数据在投影阶段就取错了位置，贴图在导出阶段被丢掉，最后在交付阶段连材质文件都没被复制过去。三段各丢一点，合起来就是「系统说成功了，打开却是一坨灰白几何」。

> **一句话**：本轮找到 9 项缺陷，其中 4 项直接决定「能不能拿到完整模型」。最致命的一项在真实数据上实测有 **1.874 倍**的像素坐标错位——纹理的每个像素都取自照片上完全不同的位置，且**没有任何报错**。这解释了为什么此前「从没得到一次完整的模型输出」。

1.1 九项缺陷一览

编号	缺陷	后果	级别
B1	M6 交付只复制 .obj，.mtl 与贴图 PNG 留在 50_tex	交付包的 OBJ 指向不存在的材质，模型无贴图	致命
B2	Open3D 写 GLB 时静默丢弃纹理与 UV	glb 在任意查看器里都是灰白，且无任何告警	致命
B3	投影用内参系 u/v 直接索引原图（实测差 1.874×）	纹理整体错位，颜色取自照片错误区域	致命
B4	照片按 float64 全量常驻内存（30 张 ≈ 9.7 GB）	真实数据在 M4 爆内存，表现为「跑到纹理阶段就没了」	致命
B5	M4 不认平铺的照片目录（00_photo 根下直接放图）	S1 解算成功但 M4 报「目录不存在」，静默降级顶点色	严重
B6	场景② 无激光几何时，唯一几何源被无条件 ×2 粗化	纯照片建模细节损失一半（voxel 0.02 → 0.04）	中
B7	场景② 网格与相机同处 SfM 坐标系，却仍做 ICP 相似变换	引入无谓旋转/缩放漂移，纹理可能整体偏移	中
B8	无色彩数据时顶点色为全 0，导出为黑色模型	查看器里整件器物发黑，看着像建模失败	中
B9	贴图存两份（_0.png 与 _diffuse.png），MTL 指向 _0.png	质检/交付按 _diffuse 找图，口径不一致且浪费一倍的图	轻

二、四项致命缺陷的证据链

2.1 B3：投影坐标错位 1.874 倍（首因）

S1 为了 COLMAP 稳定，会把超尺寸照片缩到 **2400 长边**再送进去（photos_small），因此 capture_meta.json 里的内参 fx / cx / cy 属于**缩小后**的图像坐标系。而 M4 纹理阶段读的是 00_photo 里的**原图**。旧实现把内参算出的 u/v 直接当作原图像素下标。

项目	实测值（E:\兵马俑照片\relic_001BMYyyyy）
capture_meta 内参坐标系	2399 × 1601（fx = 1803.84, cx = 1199.5, cy = 800.5）
00_photo 原图尺寸	4496 × 3000（13.49 MP，30 张）
比例	4496 / 2399 = **1.8741**
旧行为	取 img[v, u]，u/v 属 2399 系 → 颜色取自左上角 **53%** 区域
报错	**无**（越界检查用的是原图宽高，永远判定为「在界内」）

这类错误之所以最难查，是因为它**完全合法**：下标不越界、纹理不空白、覆盖率指标也正常，只是每个像素都来自照片的另一个位置。修复后 M4 的指标里新增 pixel_scale，真实数据实测输出 [1.8741]，可直接核对。

2.2 B1：交付包只有 .obj，没有材质和贴图

OBJ 的贴图不在文件本体里，而在**旁挂文件**：.obj --mtllib--> .mtl --map_Kd--> .png。旧 export_delivery 对每个格式只做了一次 shutil.copy2，把 .obj 改名搬到 80_export/，材质与贴图全部留在 50_tex/。

```
# 修复前 80_export/demo_relic/
accuracy.csv  demo_relic.glb  demo_relic.obj  demo_relic_thesis_table.csv

# 交付 obj 第 5 行（引用了一个不在本目录的材质）
mtllib demo_relic_L1.0.mtl      <- 该文件在 50_tex 里，没被复制

# 修复后
accuracy.csv  demo_relic.glb  demo_relic.mtl
demo_relic.obj  demo_relic_diffuse.png  demo_relic_thesis_table.csv
mtllib demo_relic.mtl
map_Kd demo_relic_diffuse.png
```

修复方式：跟随 OBJ 的材质引用链逐层复制，并把交付包内的相对引用统一改写为 <item>.mtl / <item>_diffuse.png；任一层缺失记 **warning** 而不再静默。顶点色 OBJ 没有 mtllib，走原路径不受影响。

2.3 B2：GLB 静默无纹理

Open3D 0.19 的 write_triangle_mesh 写 glb 时只写几何。实测产物解析 glTF JSON：

字段	修复前	修复后（trimesh 写出）
images	None（无贴图）	1 个内嵌 PNG（bufferView）
textures	None	[{source: 0}]
几何属性 TEXCOORD_0	缺失	存在
materials	[]（空）	baseColorTexture 指向贴图
baseColorFactor	—	**[1,1,1,1]**（trimesh 默认 0.4 灰，会把贴图压暗 60%）
doubleSided	—	true（薄壁/开孔处不出现黑洞）

GLB 改由 **trimesh（MIT，项目既有可选依赖）**写出：逐顶点 UV + PIL 图像 + PBRMaterial。trimesh 不可用或写出失败时**显式告警**并回退 Open3D，不再假装成功。注意 trimesh 需要**逐顶点** UV（N×2），而 Open3D 存的是逐角 UV（3F×2），导出器内部按面索引回填还原。

2.4 B4：照片按 float64 全量常驻内存

```
# 旧实现（photo_texture.project_photos_to_texture）
photos[name] = np.asarray(Image.open(p), dtype=np.float64)

# 30 张 4496×3000：
#   单张 = 4496*3000*3*8 = 323.7 MB
#   合计 = 9.71 GB  -> 16 GB 机器上叠加 open3d 必然换页/崩溃
```

修复：改为 **uint8 常驻**，并按 2 GB 总预算等比缩小（缩小因子自动被 pixel_scale 吸收，不改变投影几何），指标里记 photo_load_factor。

三、次级缺陷与修复

3.1 B5：平铺照片目录

S1 的照片解析（_photo_dir_for）允许用户把照片**直接平铺在 00_photo/ 根下**而不建 <item> 子目录；但 M4 只认 00_photo/<item>。结果是「S1 识别到 30 张、M4 说目录不存在」→ 降级顶点色 → 交付没有照片纹理的模型。现抽出 resolve_photo_dir() 与 S1 同源，两级回退（item 子目录 → 平铺根目录 → 可配置 photo_dir）。

3.2 B6：场景② 唯一几何源被无端粗化

M1 的融合权重分支以 novel 判定：重叠高 → 体素 ×2.0（降权）。但纯照片场景下 outputs 为空、novel 恒为 0，于是**唯一的几何来源也被判成「冗余」并粗化一倍**（0.02 → 0.04）。修正判据：**没有既有几何时不存在「冗余」，必须用配置原值。**

3.3 B7：对同坐标系的网格再做 ICP

纯照片场景的网格本身由 SfM 稠密重建得到，与相机位姿**同处一个坐标系**，不需要坐标系对齐。旧实现对 30 张照片一律跑 align_points_to_mesh（含尺度自由度），可能估出非 1.0 的 scale 并把网格整体缩放/旋转。现由调用方传入 geometry_is_sfm=not task.raw_paths，该分支直接跳过并在 metrics 记 status=skipped。

3.4 B8：无色彩数据导出黑色模型

无颜色点云经泊松重建后顶点色为**全 0**，OBJ 里写成 v x y z 0 0 0，查看器里整件发黑。现在无色彩来源时统一刷中性石灰灰，指标记 default_gray=True，报表可区分「几何模型」与「纹理缺失」。

3.5 B9：贴图命名双份

Open3D 固定写 <base>_0.png，而 M5 质检与交付按 <base>_diffuse.png 找图，代码里还额外保存了一份 diffuse，等于同一张图存两份、MTL 指向的那份没人用。现收敛为单一 <base>_diffuse.png：改写 MTL 的 map_Kd，删除 **_0.png。**

四、修改清单

文件	改动
src/h3d/photo_texture.py	像素比例换算（新增 pixel_scale 指标）；照片 uint8 + 2 GB 预算；resolve_photo_dir() 与 S1 同源；geometry_is_sfm 跳过对齐；贴图交由导出器统一落盘
src/h3d/texture/uv.py	export_textured(uv=, image=)；trimesh 写带纹理 GLB；OBJ 三件套命名收敛（_normalize_obj_texture）；逐顶点 UV 还原
src/h3d/texture/__init__.py	UV 路径把 uv/image 传给导出器；无色彩时中性灰 + default_gray 指标
src/h3d/publish/__init__.py	export_delivery 跟随 mtllib/map_Kd 复制旁挂文件并改写引用（_copy_obj_sidecars）
src/h3d/runner/__init__.py	M4 传 geometry_is_sfm；M1 无既有几何时不粗化唯一几何源
h3d_gui.spec	hiddenimports 增加 trimesh / trimesh.exchange.gltf / visual.material / visual.texture
tests/test_texture_export.py	新增 9 项：像素换算等价性、平铺目录解析、OBJ 三件套、GLB 纹理、交付旁挂文件

五、验证

5.1 单元测试

新增 tests/test_texture_export.py 9 项，全量 **602 项通过、0 失败**（基线 593 + 9）。其中最关键的一项是**等价性测试**：同一内参、同一场景，把照片按 2 倍最近邻放大后重新投影，要求两张纹理**逐像素一致**（容差 8/255）；这正是 B3 的判别条件——修复前该测试必然失败。

5.2 真实数据核对

核对项	结果
内参坐标系	2399 × 1601
原图尺寸	4496 × 3000
投影 pixel_scale	**[1.8741]**（与 4496/2399 一致）
照片目录解析	E:\兵马俑照片\00_photo\relic_001BMYyyyy（命中）

5.3 端到端（内置示例，16 张 640 px）

```
S1 完成：16/16 张注册，1 相机，50.2s
M1 完成：1 站，8000 -> 4363 点
M3 完成：4363 点 -> 10039 顶点 / 19829 面
M4 完成：paint_mode=photo，照片投影纹理，2 文件
M5 完成：coverage=100.0%, rmse=27.39mm -> pass（纹理 coverage 29%，ok）
M6 完成：交付包贴图随附（demo_relic.mtl + demo_relic_diffuse.png）
批量处理完成：done=1 failed=0
```

交付目录实测：demo_relic.obj + demo_relic.mtl + demo_relic_diffuse.png（1.09 MB）+ demo_relic.glb（1.61 MB）。GLB 解析确认 images / textures / TEXCOORD_0 / baseColorFactor=[1,1,1,1] 全部就位。

六、仍未解决 / 需注意

项目	说明	优先级
稠密阶段耗时	30 张 1600 长边单遍约 20 分钟、双遍约 45~50 分钟。首次跑通建议 dense_geom_consistency=false	高
纯照片模型无绝对尺度	SfM 输出为任意尺度，POI 参数（voxel 等）按绝对米制解释，仅深度 8 的泊松与 MPS 相对指标不受影响	中
交付出模的坐标系	场景① 经 ICP 对齐后网格处于 SfM 坐标系，与激光扫描的度量坐标系不同源，需在报告里注明	中
大网格纹理耗时	泊松可能产出 50 万面以上；投影为 Python 逐面循环，预计 6~15 分钟，建议设 mesh.simplify_target	中
GUI 内嵌预览	matplotlib plot_trisurf 不显示纹理，真实观感需用「导出/3D 窗口」查看（外部 Open3D 窗口支持 T 键切纹理）	低

七、下一步

代码侧已改完并通过全部测试，但**发行版 exe 仍是 r6.2**——上述修复尚未进入打包产物。若要用真实 30 张照片做端到端验证，必须重打包（H3D.exe → Inno 安装包 → 发行 zip，并同步源码快照与文档）。

> **建议的重跑配置**（首次跑通优先）：
1) 00_photo 下只保留一尊造像的照片，避免同一批照片被重复计算；
2) 稠密先关（dense: false）——S1 稀疏只要约 8 分钟，可先确认 M1→M6 全链路与贴图是否正确；
3) 确认无残留 H3D.exe / colmap.exe 进程（单实例锁会拒绝第二个实例）。